

Meeting Notes & Action Items

Vibration monitoring platform — agent work

10 September 2026
5 action items
1 point clarified

Action items at a glance

#	ACTION	AREA	EFFORT
1	Make the SQL agent dynamic — read the database structure at runtime so it survives any change to the database	SQL agent	Medium
2	Take the ISO tool out of the knowledge-base agent and make it a standalone agent of its own	KB agent	Medium
3	Verify the RPM calculation against the backend — how it is done and whether it is right	Backend	Small
4	Terminal runner with isolation — running one agent must not touch any other agent's data	Agent tooling	Medium
5	Two documents — one plain language, one technical, covering everything built so far	Documentation	Medium

1. Make the SQL agent dynamic

As agreed: read the DB structure / schema, no hardcoded columns, so that if the database changes one day it can still retrieve the data dynamically.

What this means

The agent currently carries a **fixed, hand-written list of column names** in its code. It should instead ask the database what columns exist and work from that answer.

Read the **structure** — table names, column names, types — and never bake actual values into the code. The structure is what the agent needs to build a query; the values come back fresh from the database every time.

Why it matters — this has already cost us once

This is not theoretical. `sql_agent/dataset.py` holds a hardcoded list called `CSV_COLUMNS` with about 25 names in it. Anything not on that list is silently dropped.

The shaft speed the platform had already calculated lived in a field called `metadata`, which was **not on the list**. It was thrown away in transit, and for a long time it looked as though the machine's speed was unknown — when it had been sitting in the database the whole time. **Item 3 exists because of the problem item 1 fixes.**

What is involved

- Query the database for its structure at startup instead of reading a fixed list.
- Build the query and the output columns from what comes back.
- A column added to the database appears automatically. A column removed is reported clearly, rather than causing a failure somewhere unrelated.

- Keep the exported file's column order stable so anything downstream that reads it keeps working.

2. Separate the ISO tool from the knowledge-base agent

As agreed: remove the ISO agent from the KB agent if it is present in it, and keep it as a totally separate agent.

The current situation

The ISO limit lookup is not a small add-on — it is threaded through the knowledge-base agent in five separate places:

WHERE	WHAT IT DOES
<code>kb_agent/tools_iso.py</code>	The lookup tool itself, reading the ISO limit tables
<code>agent.py</code> — question pattern	A rule that spots when a question is about ISO limits
<code>agent.py</code> — <code>_forced_iso_lookup()</code>	Forces the lookup to run in code rather than leaving it to the model
<code>agent.py</code> — <code>_iso_answer_block()</code>	Writes the limit numbers into the answer directly
<code>agent.py</code> — system prompt	Instructs the model to call the ISO tool for any limit question

Why it ended up embedded

Worth recording, because it is the reason the extraction needs care. The model was getting ISO limits wrong when left to answer from the books — reading the wrong table, quoting the wrong figure. The fix was to stop asking it and have the code look the numbers up and write them in. That works, and it is why the answers are reliable today.

Whatever replaces it must keep that property. If the separate agent goes back to letting a model read limits out of a book, the original problem returns.

What is involved

- Move the lookup into its own agent package with its own entry point, so it can be run on its own.
- Remove the forced-lookup path, the question pattern and the prompt instruction from the KB agent, so it goes back to answering purely from the books.
- Keep the limit numbers coming from the unit-tested source, not from a model.
- Decide whether the KB agent should mention that a separate ISO agent exists when someone asks it a limits question, or simply say it does not cover limits.

3. Verify the RPM calculation against the backend

As agreed: verify the RPM calculation against the backend — how it is made, how it is carried out, all of it.

The calculation as currently implemented is set out in full below, so it can be checked line by line.

The problem being solved

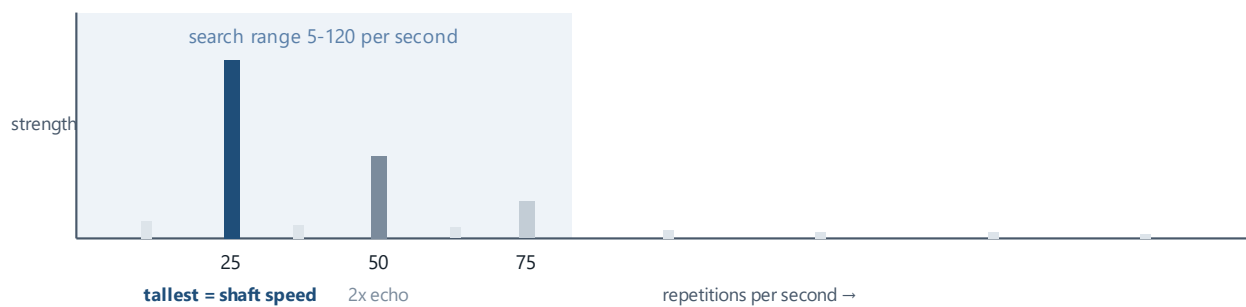
To diagnose a spinning machine you must know **how fast the shaft turns**. Every other calculation depends on it. Ideally a tachometer records it directly — the field `rotation_speed_rpm` exists for exactly this. **It is empty on all 9 captures**. So the speed is worked out from the vibration signal instead.

How the calculation works

The core idea: a spinning shaft is never perfectly balanced, so it thumps **once per turn**. That thump is the loudest repeating pattern in the signal. Find the rate of the loudest repetition and you have the shaft speed.

- 1 **Record the vibration.** For this pump: 8192 measurements taken at 25,600 per second, which is 0.32 seconds of signal, on each of 8 measuring points.
- 2 **Taper the ends of the clip.** A Hann window is applied. Cutting a recording off abruptly creates false patterns at the join, the way a song cut mid-note produces a click that was never played. Tapering the volume to zero at both ends removes them. The result is then scaled back up, because tapering also shrinks the real peaks.
- 3 **Convert the wiggle into a list of rates (FFT).** The Fourier transform takes a messy up-and-down signal and reports *which repeating rates it is built from, and how strong each one is* — like listening to a chord and naming the individual notes in it.
- 4 **Search only a sensible range.** Only rates between **5 and 120 per second** are considered — that is 300 to 7,200 rpm, the range real industrial machines run at. Everything outside is ignored, so electrical hum and high-frequency noise cannot be mistaken for the shaft.
- 5 **Take the tallest peak in that range.** That rate is the shaft speed for that measuring point.
- 6 **Convert to rpm.** Multiply by 60. Rates are per second; rpm is per minute.
- 7 **Take a vote across the 8 measuring points.** Each produces its own answer. The value most of them agree on is used, and any disagreement is recorded rather than averaged away.

What it produced for this pump



The signal broken into its repeating rates. The tallest bar inside the search range is taken as the shaft speed.

MEASURING POINT	RATE FOUND	EQUALS	VERDICT
Channels 0, 1, 2, 4, 5, 6, 7 — seven of eight	25.0 /sec	1500 rpm	majority, used
Channel 3	50.0 /sec	3000 rpm	outvoted, reported

Result: 1500 rpm, on a 7-of-8 majority.

Why channel 3 disagreed

50 is exactly double 25. A shaft that thumps once per turn usually also produces a weaker echo at twice that rate. On channel 3 that echo happened to be the taller of the two, and since both 25 and 50 sit inside the search range,

"tallest peak" picked the wrong one.

This is the known weakness of the method, and it matters: **a shaft speed wrong by a factor of two puts every calculation built on it out by the same factor.** The vote is what protects against it, and the disagreement is printed rather than hidden.

The independent check. The manufacturer's label for this pump states **1480 rpm**. The calculation gave 1500 rpm — **1.4% apart**. Nothing in the calculation reads the label, so these are two genuinely independent routes to the same answer. Good evidence the method is sound.

What to check in the backend

CHECK	WHY
Is the 5–120 per second search range right for our machines?	It sets what speeds can ever be found: 300 to 7,200 rpm. A machine outside it would be reported wrongly.
Should the tallest peak win, or should a harmonic check run first?	Exactly what went wrong on channel 3. Checking whether a candidate is double another candidate would catch it before the vote.
Is the sampling rate stored per capture correct?	The whole scale depends on it. A wrong recorded rate makes every speed wrong by the same proportion.
Can <code>rotation_speed_rpm</code> be populated at capture time?	A real tachometer reading would replace the estimate entirely and remove this uncertainty.
Should the nameplate speed be used as an automatic sanity check?	It is already in the database. An estimate far from the nameplate is worth flagging.

4. Terminal runner with agent isolation

As agreed: a different agent running script for the terminal, so that if we want a single agent's output it does not alter any other agent's data or do anything else.

The requirement

Isolation is the point here, more than convenience. Running one agent must leave every other agent's data exactly as it was. Asking the knowledge-base agent a question should not touch the SQL agent's exports; running the report agent should not rebuild a search index.

What is involved

- One entry point that takes the agent name and its arguments, and runs only that agent.
- Each agent writes only inside its own output area. No shared file that two agents both write to.
- Read-only where possible: an agent that only needs to read platform data should not be able to write to it.
- Nothing loads unless it is needed — running the SQL agent should not pull in the book index.
- State that would collide between agents gets separate storage, so one run cannot overwrite another's results.

Related to item 2: separating the ISO tool out of the KB agent makes this easier, because each agent then owns a clearly separate job.

5. Two documents

As agreed: create two documents – one in plain language and one for technical understanding – about everything that has been done.

DOCUMENT	AUDIENCE	CONTAINS
Plain language	Presentation — management, non-technical reviewers	What has been built, what it found, what it still needs. No code, no formulas. Technical terms explained in ordinary words wherever they cannot be avoided.
Technical	Engineers, whoever maintains this next	Architecture, how data moves between the agents, the calculations, file and function names, known limitations, how to run and extend it.

Both cover the same ground: the SQL agent, the knowledge-base agent, the report agent, the calculations, and the data problems found. Best written after items 1–4, so they describe the finished state rather than needing revision.

6. Point clarified — the healthy baseline

Question raised: if it is taking the same file and calling it healthy, tell it to consider a different file.

The direction is right; one detail changes the answer

Yes — the baseline must come from a different recording. That is exactly the problem. **But there is no different file to point it at.**

FILE NAME	TIMES UPLOADED	DISTINCT SETS OF NUMBERS
sample_vibration_data (1).csv	7	1
sample_vibration_data.csv	2	1
All nine uploads together	9	1

The two file names hold **identical numbers**. Switching from one to the other changes nothing at all — same data, different name.

What a baseline is, in plain terms

Think of it as a **"before" photograph**. You photograph a wall when it is freshly painted. Months later you photograph it again and compare, and the difference tells you how much it has faded.

The baseline is that "before" photo of the machine — a recording made while it was running normally. Every later recording is compared against it to answer one question: *is this different from when it was healthy?*

What is happening now: we are comparing today's photo against today's photo. The same image on both sides. The answer comes back "no change — perfect" every time. Not because the machine is fine, but because no "before" photo was ever taken.

Why no software change can fix it

The software does not decide anything is healthy. **A person does**, by marking one recording as the healthy one. The software simply trusts that and compares against it.

So what is needed is not a code change and not a different file name. It is **a fresh recording of the pump taken while it is running normally** — a genuinely new measurement, on a different day. That becomes the "before" photo.

Until then, "not checked" is the honest answer for those readings. Marking the current file as the healthy one would report them as **normal**, which reads as *we checked and it is fine* when nothing was actually checked.

Suggested order

1. **Item 3 — verify the RPM calculation.** Smallest, and the findings feed the technical document.
2. **Item 1 — make the SQL agent dynamic.** Removes the class of bug that hid the shaft speed; everything downstream benefits.
3. **Item 2 — separate the ISO tool.** Clarifies what each agent is responsible for, which makes item 4 simpler.
4. **Item 4 — the terminal runner.** Best built once items 1–3 have settled what it is running.
5. **Item 5 — the two documents.** Written last, describing the finished state.

Item 6 is not on this list because it is not development work. It needs a fresh measurement taken at the plant, and can happen in parallel at any time.

Meeting notes and action items · vibration monitoring platform · 10 September 2026.

Five action items recorded, one point clarified. The RPM section describes the calculation as currently implemented in the backend, read from the source at the time of writing; the file and upload figures were queried from the running database.